

PEPS structure documentation

Erik Lennart Weerda

March 2026

Contents

1	General mathematical task and notation	1
1.1	Schrödinger equations and structure of the vector spaces	1
1.2	Aside: Graphical notation for tensor networks	2
1.3	PEPS as a parameterized sub-manifold	3
1.4	Approximate Schrödinger evolution of the PEPS-state	4
1.5	Sampling the computational basis	4
2	Necessary computational tasks and their relations	5
2.1	Evaluating coefficients $\Psi_{\mathbf{S}}$	6
2.2	Evaluating derivatives $\frac{\partial \Psi_{\mathbf{S}}(\theta)}{\partial \theta_i}$	6
2.3	Solving for $\dot{\theta}$ and integrating the evolution equation	7
2.4	Generating samples $ \mathbf{S}\rangle$ and probabilities $p_{\Psi}(\mathbf{S})$	8
3	Workflow & typical problem sizes	8
3.1	Relevant numbers for the ansatz:	8
3.2	Workflow	8
3.3	Imaginary-time evolution	9
3.4	Real-time evolution	9
4	Algorithms	11
4.1	The boundary-MPS algorithm	11
4.1.1	Compression algorithms	12

1 General mathematical task and notation

1.1 Schrödinger equations and structure of the vector spaces

The general aim of our PEPS code is to evolve a vector in a complex vector space $|\Psi\rangle \in \mathcal{H} = \bigotimes_{i=1}^n \mathbb{C}_i^d$ according to an evolution equation - in our case, the Schrödinger equation

$$i \frac{\partial}{\partial t} |\Psi\rangle = H |\Psi\rangle, \quad (1)$$

or its imaginary-time cousin

$$\frac{\partial}{\partial \tau} |\Psi\rangle = -H |\Psi\rangle. \quad (2)$$

The complex vector space $\mathcal{H}$ is a tensor product of n complex vector spaces of dimension d . Each one of these n vector spaces corresponds to one of the degrees of freedom of our system. These degrees of freedom should be imagined as physically arranged in a finite two-dimensional lattice of size (L_x, L_y) - we will make use of this spatial structure in the following.

In the evolution equations, H is a hermitian matrix acting on $\mathcal{H}$. We assume $H(t) = H$ is time independent. The size of the complex vector space $\dim(\mathcal{H}) = d^n$ is too large to do exact computations for our typical sizes of $n \sim 15 \times 15 = 225$, where even storing a general vector $|\Psi\rangle$ in an arbitrary basis is unfeasible. In order to still make progress, we restrict the vector $|\Psi\rangle$ to a parameterized sub-manifold of the full complex vector-space $\mathcal{H}$. This restriction to the parameterized sub-manifold is physically motivated and leverages the local arrangement of the degrees of freedom.

To give an example of this, let us restrict to local vector spaces of size $d = 2$ and let us call the basis vectors

$$|\mathbf{S}\rangle = |s_{(1,1)} \dots s_{(L_x, L_y)}\rangle := |s_{(1,1)}\rangle \otimes \dots \otimes |s_{(L_x, L_y)}\rangle \quad (3)$$

of $\mathcal{H}$ the many-body configurations $|\mathbf{S}\rangle$ of the system. Here $|s_{(1,1)}\rangle$ is a basis vector of one of the spatially arranged local vector spaces $\mathbb{C}^2$, and we refer to this as the *computational basis*. We expand the general vector $|\psi\rangle$ in this computational basis

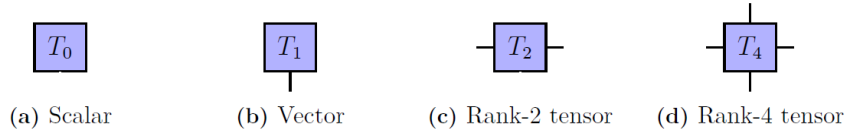
$$|\Psi\rangle = \sum_{\{\mathbf{S}\}} \Psi_{\mathbf{S}} |\mathbf{S}\rangle, \quad (4)$$

and will make assumptions about the structure of the coefficients $\Psi_{\mathbf{S}} = \langle \mathbf{S} | \psi \rangle := |\mathbf{S}\rangle^\dagger |\Psi\rangle$ to achieve our desired restriction to a parameterized sub-manifold.

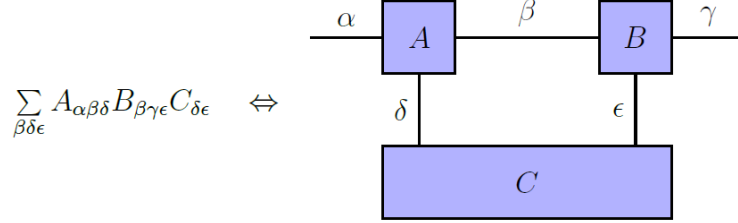
Edoardo: Could one say that the $|\mathbf{S}\rangle$ represents a system on n 2-state spin particle? $\rightarrow$ Yes, you can think of it as a single basis state describing one of the 2^n possible configurations of the system.

1.2 Aside: Graphical notation for tensor networks

Tensors of rank n , are represented using a shape (circle, square, or something else) with n legs. Each of these legs represents one of the indices of the tensor. Here are some examples:



Summing over indices of multiple tensors (contraction) is represented by connecting the legs corresponding to the indices that we are summing over. An example, where we show the indices explicitly, is the following:



Lastly, in the following, if we are fixing one of the indices of a tensor to a specific value (e.g., $i = 1$) we represent this graphically by adding a little red ball to that index.

1.3 PEPS as a parameterized sub-manifold

The projected-entangled pair states (PEPS) - making up our sub-manifold in which we are considering $|\Psi\rangle$ during the evolution, are defined by their basis coefficients of the form

$$\Psi_S = \quad (5)$$

This form reduces the number of independent basis coefficients of the vector $|\Psi\rangle$ from d^n to $n_p := \sum_{(i=1, j=1)}^{(L_x, L_y)} \#T[i, j]$, where $\#T[i, j]$ is the number of entries in the local tensor $T[i, j]$, located at lattice positions $(x = i, y = j)$.

Note:

The number of independent coefficients n_p in a PEPS-vector is linear in n – as opposed to exponential in the general case.

As a matter of notation, we use θ as the parameters of the PEPS-vector (the entries of the tensors $T[i, j]$), such that we write $|\Psi(\theta)\rangle$ to signify the parameterized PEPS-vector.

Edoardo: Am I undersyandng correctly that θ is an array of size n_p ? $\rightarrow$ Yes.

1.4 Approximate Schrödinger evolution of the PEPS-state

The *central aim* of the PEPS-code is to evolve a vector $|\Psi(\theta)\rangle$ of the PEPS-submanifold of $\mathcal{H}$ according to the Schrödinger equations (1) and (2).

Note:

As the number of parameters n_p in our PEPS-vector $|\Psi(\theta)\rangle$ is smaller than the dimension of the vector space $\dim(\mathcal{H})$ the Schrödinger equation will not have an exact solution on our sub-manifold!

As we are evolving only the parameters θ , to stay on our PEPS-sub-manifold, we rewrite the evolution equation (here for concreteness, the imaginary-time equation)

$$|\dot{\Psi}(\theta)\rangle := \frac{\partial}{\partial \tau} |\Psi\rangle = -H |\Psi(\theta)\rangle \quad (6)$$

$$\frac{\partial |\Psi(\theta)\rangle}{\partial \theta} \dot{\theta} = -H |\Psi(\theta)\rangle \quad (7)$$

We now reexpress this equation with i indexing the parameters θ , stating that the equation is true for every element of the computational basis of $\mathcal{H}$

$$\sum_i \underbrace{\frac{\partial \langle \mathbf{S} | \Psi(\theta) \rangle}{\partial \theta_i}}_{=: \tilde{O}_{\mathbf{S},i}} \dot{\theta}_i = - \underbrace{\langle \mathbf{S} | H | \Psi \rangle}_{=: \tilde{E}_{\mathbf{S}}} \quad \forall \mathbf{S} \quad (8)$$

$$\tilde{O} \dot{\theta} = -\tilde{E}. \quad (9)$$

Here we have defined the matrix $\tilde{O}$ of size $\dim(\mathcal{H}) \times n_p$ with elements $\tilde{O}_{\mathbf{S},i}$ and the vector $\tilde{E}$ with elements $\tilde{E}_{\mathbf{S}}$ for later convenience.

As noted above, if the number of parameters n_p is smaller than the dimension of our vector space, the above equations will, in general, not have an exact solution. Instead, we are looking for the best approximation of a solution on our PEPS-state submanifold

$$\min_{\dot{\theta}} \sum_{\mathbf{S}} \left\| \sum_i \tilde{O}_{\mathbf{S},i} \dot{\theta}_i + \tilde{E}_{\mathbf{S}} \right\|^2. \quad (10)$$

1.5 Sampling the computational basis

Instead of finding the best approximation on the entire Hilbert space, we use Monte Carlo sampling to find a solution on the relevant, sampled part of the

Hilbert space whose dimension is the number of samples n_s

$$\min_{\dot{\theta}} \sum_{\mathbf{S}} p_{\Psi}(\mathbf{S}) \left\| \sum_i \underbrace{\frac{\tilde{O}_{\mathbf{S},i}}{\Psi(\mathbf{S})}}_{=:O_{\mathbf{S},i}} \dot{\theta}_i + \underbrace{\frac{\tilde{E}_{\mathbf{S}}}{\Psi(\mathbf{S})}}_{=:E_{\mathbf{S}}} \right\|^2. \quad (11)$$

This is solved by restricting O and E to the basis states $|S\rangle$ drawn as samples and isolating $\dot{\theta}$

$$\boxed{\dot{\theta} = -(O^\dagger O)^{-1} O^\dagger E} \quad (12)$$

or

Edoardo: Stated like this, it is hard to understand why $\dot{\theta}$ can be expressed in one of these two ways. I suggest to show the derivation of one of them. Perhaps it is sufficient to say that this is a least square problem which is solved taking the first derivative of the objective function with respect to the minimization parameter and putting it equal to zero. The result is a linear system which can be formally represented as inverting O . Because O is not square then one has to use the definition of pseudo-inverse (or Moore-Penrose inverse) of $O \rightarrow$ the idea is exactly that! We are looking for a zero point of the function of which we are taking the absolute value squared. This minimizes the minimization problem and reduces our problem to solving the equation $-O\dot{\theta} = E$. As O is non-square, we use the Moore-Penrose inverse. Depending on whether we use the left or the right inverse, we obtain equation (12) or (13).

$$\boxed{\dot{\theta} = -O^\dagger (OO^\dagger)^{-1} E} \quad (13)$$

Which of the two equations above is more efficient and hence should be used depends on whether the number of parameters n_p is larger than the number of samples n_s drawn, cf. Sec.... . Using $\dot{\theta}$, we can use our favorite integration method to get the updated version of our parameters θ such that they follow approximately the flow of the Schrödinger equation.

2 Necessary computational tasks and their relations

To use the mathematical setup in Section 1, we need to be able to perform only a few tasks numerically. It is further important to note that several of these numerical tasks involve identical steps, which allows us to use intermediate results for multiple of these numerical task.

2.1 Evaluating coefficients $\Psi_{\mathbf{S}}$

In order to set up the equations (12) or (13) to define $\dot{\theta}$, we need to be able to calculate the elements $\tilde{E}_{\mathbf{S}} = \langle \mathbf{S} | H | \Psi \rangle$. We are dealing with Hamiltonian operators $H = \sum_i h_i$, where h_i acts non-trivially only in a local region (e.g. on neighboring sites of our two-dimensional lattice). Therefore we have

$$\tilde{E}_{\mathbf{S}} = \langle \mathbf{S} | \sum_i h_i | \Psi \rangle = \sum_i \sum_{\mathbf{S}'} (h_i)_{\mathbf{S}'\mathbf{S}} \langle \mathbf{S}' | \Psi \rangle = \sum_i \sum_{\mathbf{S}'} (h_i)_{\mathbf{S}'\mathbf{S}} \Psi_{\mathbf{S}'}. \quad (14)$$

Hence, the numerical tasks we need to be able to perform in order to evaluate $\tilde{E}_{\mathbf{S}}$ are evaluating matrix elements of our Hamiltonian terms $(h_i)_{\mathbf{S}'\mathbf{S}}$ (which is a trivial task) and evaluating coefficients $\Psi_{\mathbf{S}'}$.

Note:

In principle, the sum over $\mathbf{S}'$ runs over all basis vectors in the computational basis. However, as the terms of the Hamiltonian act non-trivially only on a few sites of our two-dimensional system, the matrix elements $(h_i)_{\mathbf{S}'\mathbf{S}}$ are zero unless the basis vectors $|\mathbf{S}\rangle$ and $|\mathbf{S}'\rangle$ differ on the small support of h_i .

The coefficients $\Psi_{\mathbf{S}'}$ are two-dimensional single-layer tensor networks, as is illustrated in Eq. 5. Exactly performing the contraction is prohibitively expensive (memory cost is exponential in linear-system size). We therefore perform the contraction approximately. For this task, we use the *boundary-MPS* algorithm. We explain this algorithm in Sec.4.1.

Code:

`generate_Oks_and_Eks(...)` : This function returns a function that calculates the entries of the matrix O and the vector E simultaneously. The returned function depends if we want to use single or multi-threading etc. It effectively performs the tasks of Sec. 2.1 and Sec. 2.2.

2.2 Evaluating derivatives $\frac{\partial \Psi_{\mathbf{S}}(\theta)}{\partial \theta_i}$

To get the elements of $\tilde{O}_{\mathbf{S},i}$ to set up the equations for $\dot{\theta}$, we need to be able to evaluate the derivative of the coefficients $\Psi_{\mathbf{S}}$ w.r.t. the parameters of the PEPS vector θ – i.e the entries of the tensors making up their network, cf. Eq. 5.

This is done by simply applying the chain rule. We first evaluate the derivative of the tensor network w.r.t. a single tensor and then the derivative of the single tensor w.r.t. one of its entries. The former is simply a two-dimensional

tensor network with one of its tensors removed

$$\frac{\partial \Psi_{\mathbf{S}}(\theta)}{\partial T[3, 2]} = \text{Diagram} \quad (15)$$

The latter is a projector onto the element of the tensor we are considering.

Hence, we can evaluate the relevant entries $\tilde{O}_{\mathbf{S},i} = \frac{\partial \Psi_{\mathbf{S}}(\theta)}{\partial \theta_i}$ by evaluating the entries of the tensor network in Eq. 15. To perform this evaluation we will again make use of the *boundary-MPS* algorithm, cf. Sec.4.1.

Note:

For a given coefficient $\Psi(\mathbf{S})$, cf. Eq. 5 we highlight that the network involved is ... same as in Eq. 5 (almost) refer to Workflow later to explain reusement

2.3 Solving for $\dot{\theta}$ and integrating the evolution equation

Here I am presenting the procedure Eq. (13), the procedure for Eq. (12) works along similar lines.

We need to update the parameters θ in our PEPS vector according to the flow of the Schrödinger equations. For this change with (imaginary-) time, we obtained:

$$\dot{\theta} = -O^\dagger(OO^\dagger)^{-1}E \quad (16)$$

This is usually solved by defining $A = OO^\dagger$ and then first solving $Ax = E$. By multiplying with $-O^\dagger$ we then obtain $\dot{\theta} = -O^\dagger x$.

The linear problem can be solved via pseudo-inverse or using a Cholesky factorization.

Code:

To evolve the parameterized vector $|\Psi(\theta)\rangle$, i.e., perform the tasks in Sec. 2.3 and update $|\Psi(\theta)\rangle$, we use the function `evolve(...)`. In the repository we are linking to here, the `evolve` function is explained in the `README.md`.

2.4 Generating samples $|\mathbf{S}\rangle$ and probabilities $p_{\Psi}(\mathbf{S})$

This task is described in some detail in Appendix B of our paper. It again involves the *boundary-MPS* algorithm, cf. Sec. 4.1.

Code:

For the task in Sec. 2.4 we create the double-layer environments using `generate_double_layer_envs(...)`. Once we have the double-layer environments we use the function `get_sample(...)` to generate a sample $|\mathbf{S}\rangle$ and its associated probability weight $p_c(\mathbf{S})$.

3 Workflow & typical problem sizes

In general, we are going to consider PEPS-vectors in which all indices, which are contracted to obtain $\Psi_{\mathbf{S}}$, cf. Eq. 5, are of equal size. We call the size of these indices the bond-dimension D . As this bond dimension controls the accuracy of our calculation, we want this to be as large as possible.

3.1 Relevant numbers for the ansatz:

1. In practice, we are usually limited to typical values of $D \in \{2, \dots, 8\}$.
2. For most calculations, the local vector spaces are of size $d = 2$, but for some cases we get up to $d = 8$.

Edoardo: Is there any relation between D and d ? $\rightarrow$ Not really. d is fixed by the problem at hand (the Hilbert space we are dealing with), whereas D controls the accuracy of our approximation/ n_p .

3.2 Workflow

1. Start by calculating the double-layer environments, necessary to generate samples $|\mathbf{S}\rangle$ and corresponding
2. IN PARALLEL: Once we have the double-layer environments, use them to independently generate $E_{\mathbf{S}}$ and $O_{\mathbf{S},i}$.
3. Using $E_{\mathbf{S}}$ and $O_{\mathbf{S},i}$ solve the linear system to obtain $\dot{\theta}$.
4. Integrate to obtain the next parameters θ of the ansatz.

Edoardo: Can you please explain what you mean with "double layer environment"? $\rightarrow$ Sorry! This is necessary for the sampling and refers to section 2.4 - I can go into more details if needed.

3.3 Imaginary-time evolution

1. Here we are typically treating systems 8×8 up to 32×32 .
2. The number of samples n_s that we need is usually quite small:

$$n_s \sim 1000 - 5000.$$

At the later stages of the evolution, we usually use more samples than in the beginning.

Because usually the number of samples is much smaller than the number of parameters in our ansatz, $n_s \ll n_p$, we typically use Eq. 12 to isolate $\hat{\theta}$. This also means that the size of the linear systems we need to solve for the task described in Sec. 2.3 is quite small and does not grow with system size; this is usually not a bottleneck.

Most of the computing time is used in the creation of the double-layer environments, which are necessary for generating samples and for evaluating $E_{\mathbf{S}}$ and $O_{\mathbf{S},i}$. As we increase the bond dimension D , the time-share for creating the double-layer bond dimensions increases.

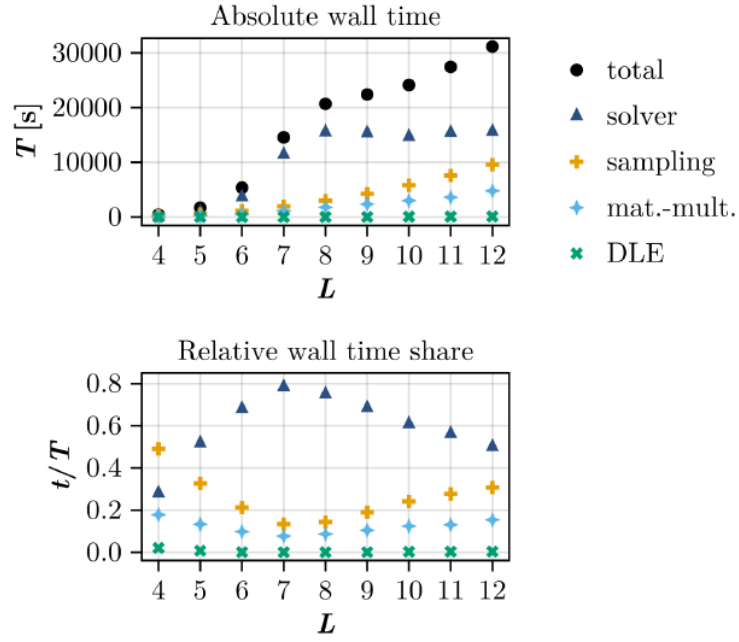
3.4 Real-time evolution

For the evolution according to the real-time Schrödinger-equation, cf. Eq. 1, we have to be much more accurate in our approach.

Edoardo: Does that mean that now you solve (13)? $\rightarrow$ this depends on whether n_s or n_p is larger. This will determine if (12) or (13) is more efficient.

1. This is why we are typically treating smaller lattices of size $(Lx \times Ly) \sim 8 \times 8$. We again have the bond dimension to control the accuracy with which we can follow the Schrödinger evolution.
2. To be more accurate we typically use a large number of samples $n_p \sim n_s$

Because we use so many samples, and all of these samples can be created from the same double-layer environment, the relative time-share of creating the double-layer environments is quite small. Instead solving the linear equations (task 2.3) and evaluating $E_{\mathbf{S}}$ and $O_{\mathbf{S},i}$ (task 2.1 and 2.2) become dominant. Below we show an example of the absolute and relative times of the individual tasks for different lattice sizes at $n_s = 4 \cdot 10^4$:

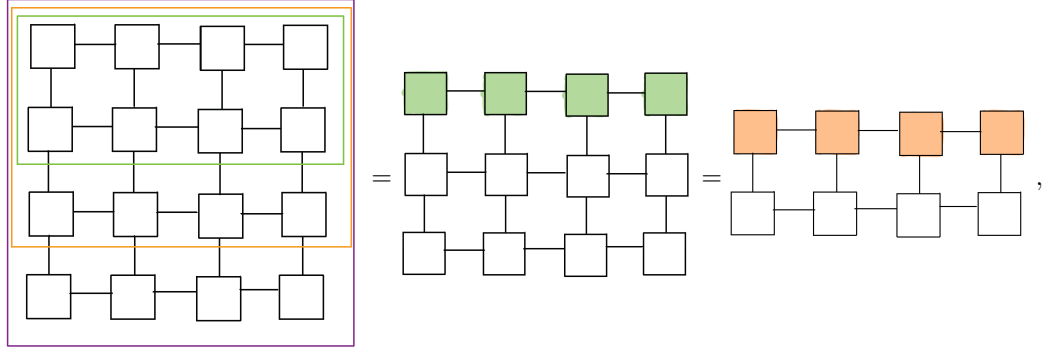


(b) sample size $n_s = 4 \cdot 10^4$

4 Algorithms

4.1 The boundary-MPS algorithm

The boundary-MPS algorithm refers to an approximate contraction scheme for a two-dimensional tensor network. Let us say all indices in this tensor network are of bond-dimension D . Concretely, in this scheme, the rows of the tensor network are contracted one after another



(17)

in this case, at first, the uppermost rows (in the **green box**) are contracted along the vertical bonds. The result is the row of green tensors. This row of green tensors has bond dimension D^2 on the horizontal bonds. As this bond dimension quickly grows out of bounds if we contract more and more rows, we use compression algorithms (cf. Sec. 4.1.1) to fix the horizontal bond dimension to a maximum value D_c , which we refer to as the *contract bond dimension*.

Note:

The computational cost of a single step, in which we are taking a boundary row with contract bond dimension D_c and absorbing a row of tensors of bond dimension D with subsequent compression of the resulting bond dimension $D_c \cdot D \xrightarrow{\text{compress}} D_c$ down to D_c again scales as $\mathcal{O}(D_c^3 D^3) + \mathcal{O}(D_c^2 D^4)$.

Edoardo: The complexity analysis in the box above is not entirely obvious to me. I am assuming the first term are the contractions while the second term is the compression. Even so, I cannot completely make sense of the exponents. Could you please elaborate? Sorry for being dense. → this scaling is explained explicitly in the clickable links below.

We define $E^u[i]$ to be the approximation of the uppermost i rows of the tensor network with the above procedure. We define $E^l[i]$ to be the approximation of the lowermost rows up to row i . Hence, we can approximate the entire tensor network as $E^u[i] \cdot E^l[i+1]$, $\forall i \in [1, L_x - 1]$.

Note:

Long thin TN are efficiently contractible...

Note:

The boundary-MPS algorithm can, of course, also be used to approximately contract double-layer tensor networks. In this case the computational cost scales as $\mathcal{O}(D_c^3 D^4) + \mathcal{O}(d D_c^2 D^6)$, where d is the dimension of the local vector spaces, and the bond dimension of the indices connecting the two layers.

4.1.1 Compression algorithms

1. Compression using SVD
2. Compression using Eigenvalue Decomposition